When Django loads your app, it looks for admin.py files the same way it looks for models.py files. To add a `Model` to the Django Admin, you create a class that inherits from `admin.ModelAdmin` and call the `register()` function to associate the class with the model. The `register()` function can also be used as a class decorator. This is my preference, as it keeps the registration close to the `admin.ModelAdmin` class declaration.

Each registered `admin.ModelAdmin` class gets a set of *CRUD pages*. CRUD, besides being a very fun acronym, stands for *create, read, update, and delete*, which refers to the actions you take on most pieces of data. The *Listing page* acts as one of the Read pages, showing all the objects for the given `Model` class. Each listed object is a link, taking you to a *Change page*, where you can update the fields of the given object. The Django Admin also has links to an *Add page*, which allows you to create new objects, as well as a *Delete page*, which—as you might have guessed—allows you to delete existing objects. Figure 5.4 shows the relationship between the `Model` and `admin.ModelAdmin` classes as well as the resulting Django Admin screens.

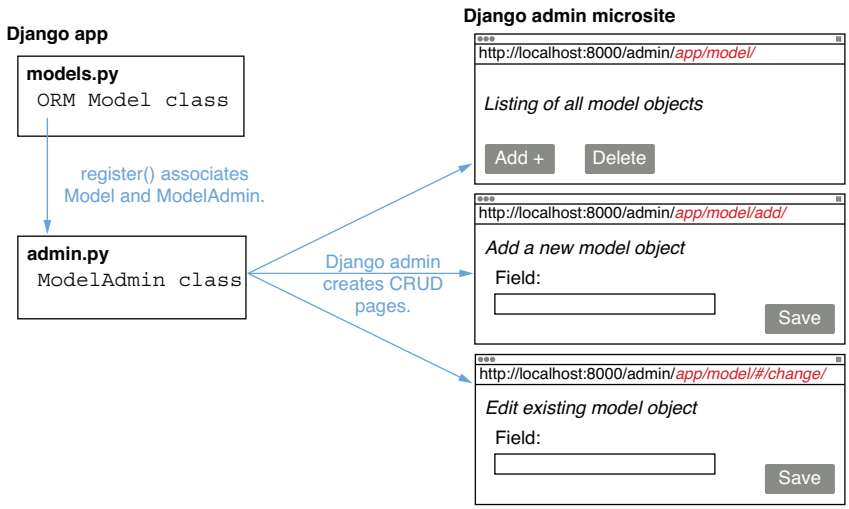


Figure 5.4 `Model` class, `admin.ModelAdmin` class, and associated Django Admin screens

Let's look at a concrete example and register the `Musician` ORM class with the Django Admin. To do this, you create a class that inherits from `admin.ModelAdmin` inside admin.py. I typically give mine the same name as the ORM class, adding `Admin` as a suffix—`MusicianAdmin`, in this case. The default behavior of the `admin.ModelAdmin` class includes all the CRUD pages, so your new class can have an empty body, using just the `pass` keyword. The skeleton admin.py file generated by `startapp` imports `django.contrib.admin` for you, and the `register()` function lives inside that module. I use `register()` as a class decorator, passing in the `Musician` ORM class itself to